

Práctica 3: Listas Doblemente

Enlazadas

Navegación bidireccional

AED I — Bloque I

2 h en clase + ~2 h en casa · Python

Algoritmos y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

¿Para qué sirve esto?

¿Quién ha usado CTRL+Z alguna vez?

- Historial de Navegador (esto lo vamos a revisar hoy)
- Spotify
- OS caché
- LinkedHashMap

¿Qué vamos a hacer?

1. Repasar la lista simple y damos el salto a la lista doble
2. Programamos juntos, en directo, una `ListaDoble` completa
3. Resolvemos el reto del palíndromo con dos punteros
4. Vemos la arquitectura de un historial de navegador con atrás/adelante.
5. Cerramos con un test individual sobre lo trabajado hoy

Objetivos de la práctica

1. Entender por qué un segundo puntero cambia la complejidad de lo que una lista enlazada puede hacer.
2. Implementar desde cero una lista doblemente enlazada en Python, comprobada con su invariante y tests.
3. Aplicar la navegación bidireccional para resolver problemas clásicos (p. ej.: palíndromo).
4. Extender una lista doblemente enlazada para resolver el historial de navegación web.
5. Razonar sobre la memoria en Python: ciclos de referencias y recolector de basura.

Flujo de la sesión

Fase	Duración	Actividad
Motivación y repaso	20 min	Ejemplos reales y punteros
Ejercicio 1: Programar lista doble	30 min	Live coding en Python + tests
Ejercicio 2: Implementar ¿Es Palíndromo?	15 min	Implementar, probar y comparar
Ejercicio 3: Operaciones adicionales	10 min	Discutir operaciones
Ejercicio 4: Historial de navegador	15 min	Discutir estructura y operaciones
Cierre y Reflexión	20 min	Tarea para casa, test individual

Repaso: La lista simplemente enlazada

En una lista simplemente enlazada, cada nodo guarda un dato y un único puntero al **siguiente** nodo. Solo se puede recorrer en una dirección, desde la `cabeza` hasta que un nodo apunta a `None`.

cabeza -> [A|•] -> [B|•] -> [C|/]

Eliminar el **último** nodo: no basta con tener el puntero a `cabeza`. Hay que recorrer toda la lista para encontrar el penúltimo nodo y hacer que su `siguiente` sea `None`. **O(n)**.

Insertar antes de un nodo dado: mismo problema. Para enlazar el nodo anterior hay que buscarlo desde la cabeza. **O(n)**.

De simple a doblemente enlazada

Lista Simplemente Enlazada:

cabeza -> [A|•] -> [B|•] -> [C|/]

Lista Doblemente Enlazada:

cabeza -> [/|A|•] <-> [•|B|•] <-> [•|C|/] <- **cola**

Eliminar el último nodo

¿Cuánto "cuesta"?

$O(1)$

¿Cómo?

1. `nodoABorrar = cola`
2. `cola = cola.anterior`
3. `cola.siguiente = None`
4. `nodoABorrar.anterior = None`

Insertar X al inicio

`cabeza -> [A <-> B <-> C] <- cola`

1. `X.siguiente = cabeza` (X apunta a A)
2. `X.anterior = None`
3. `cabeza.anterior = X` (A ahora conoce a X)
4. `cabeza = X`

`cabeza -> [X <-> A <-> B <-> C] <- cola`

Caso especial Lista vacía: `cabeza = cola = X`

Eliminar un nodo en general

$\text{cabeza} \rightarrow [A \leftrightarrow B \leftrightarrow C] \leftarrow \text{cola}$

Si tenemos el puntero a B, eliminar B de la lista.

1. $A.\text{siguiente} = B.\text{siguiente}$ (A apunta a C)

2. $C.\text{anterior} = B.\text{anterior}$ (C apunta a A)

Si B es la cabeza: $\text{cabeza} = B.\text{siguiente}$

Si B es la cola: $\text{cola} = B.\text{anterior}$

Si B es el único elemento: $\text{cabeza} = \text{cola} = \text{None}$

$\text{cabeza} \rightarrow [A \leftrightarrow C] \leftarrow \text{cola}$

El invariante de la lista doble

Lo que SIEMPRE debe ser cierto al terminar cualquier operación:

- `tamano == 0` $\Leftrightarrow$ `cabeza is None` $\Leftrightarrow$ `cola is None`
- `cabeza.anterior is None` y `cola.siguiiente is None`
- de cabeza a cola por `.siguiiente` hay exactamente `tamano` nodos
- `x.anterior.siguiiente is x` para todo nodo salvo la cabeza

`invariante_correcto()` ya está en la plantilla: los tests lo comprueban después de **cada** operación.

«invariante roto tras `delete_last`» = el resultado parece bueno, pero algún puntero ha quedado mal por dentro.

Cómo trabajamos: plantilla y tests

Completa lista_doble.py en orden y ejecuta el grupo de tests de cada paso:

```
cd python
python3 -m unittest -v test_lista_doble.PruebasInsercion

PruebasInsercion          insert_last, __reversed__
PruebasInsercionRelativa  insert_before, insert_after
PruebasBorrado            delete_node, delete_first, delete_last
PruebaContraList          2000 operaciones contra una list (oráculo)
PruebasPalindromo         es_palindromo
```

Python por debajo: `len(l) → __len__ · for x in l → __iter__ · reversed(l) → __reversed__`

`None ≈ nullptr · compara nodos con is · borrar en lista vacía → IndexError (como list.pop())`

Ejercicio 1: Implementando Lista Doble (1)

¡Abrimos nuestro editor!

- Clase Nodo con 2 punteros:

```
class NodoDoble:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None
        self.anterior = None
```

Ejercicio 1: Implementando Lista Doble (2)

- Clase Lista Doble:

```
class ListaDoble:  
    def __init__(self):
```

- Operaciones elementales:

```
    def insert_first(self, dato):  
    def insert_last(self, dato):  
    def print_forward(self):  
    def print_backward(self):  
    def __iter__(self):    # for x in l  
    def __reversed__(self): # reversed(l)
```

cabeza -> [A <-> B <-> C] <- cola

Ejercicio 1: Implementando Lista Doble (3)

- Inserción general:

```
def insert_before(self, nodo, dato):
```

```
def insert_after(self, nodo, dato):
```

cabeza -> [A <-> B <-> C] <- cola

Ejercicio 1: Implementando Lista Doble (4)

- Borrando nodos:

```
def delete_first(self):
```

```
def delete_last(self):
```

```
def delete_node(self, nodo):
```

cabeza -> [A <-> B <-> C] <- cola

Ejercicio 2: Implementar ¿Es Palíndromo? (1)

cabeza -> [1 <-> 2 <-> 3 <-> 2 <-> 1] <- cola



cabeza -> [1 <-> 2 <-> 3 <-> 4 <-> 5] <- cola



```
def es_palindromo(self):
```

- Recorremos desde cabeza y cola a la vez. ¿Basta con tamaño/2?

Ejercicio 2: Implementar ¿Es Palíndromo? (2)

```
def es_palindromo(self):  
    if self.tamano <= 1:  
        return True  
  
    inicio = self.cabeza  
    fin = self.cola  
  
    for _ in range(self.tamano // 2):  
        if inicio.dato != fin.dato:  
            return False  
        inicio = inicio.siguiente  
        fin = fin.anterior  
  
    return True
```

Otras operaciones

- Intercalar (2 listas)

[1 <-> 3 <-> 5]

[2 <-> 4 <-> 6]

Resultado: [1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6]

- Invertir (reverse)

[1, 2, 3] -> [3, 2, 1]

- Rotar (n posiciones)

[1 <-> 2 <-> 3 <-> 4] rotar 2->[3 <-> 4 <-> 1 <-> 2]

Manejando el historial de un navegador (1)

Historial: leetcode.com ↔ google.com ↔ facebook.com ↔ youtube.com

^

actual (estás en facebook.com)

visit("linkedin.com") desde aquí:

1. youtube.com se descarta (estaba "por delante")
2. se enlaza un nuevo nodo linkedin.com justo después de facebook.com
3. actual pasa a apuntar a linkedin.com

Historial: leetcode.com ↔ google.com ↔ facebook.com ↔ linkedin.com

^

actual

Manejando el historial de un navegador (2)

- No necesitamos ninguna estructura de datos nueva. Solo donde estamos ahora.
- Un único puntero 'actual', sobre una lista doble.
- Tres operaciones
 - Visit
 - Back
 - Forward

Manejando el historial de un navegador (3)

Traza de navegación:

```
BrowserHistory("leetcode.com")  
visit("google.com")
```

```
visit("facebook.com")  
visit("youtube.com")
```

```
back(1)    -> "facebook.com"  
back(1)    -> "google.com"  
forward(1) -> "facebook.com"  
visit("linkedin.com")
```

```
forward(2) -> "linkedin.com"  
back(2)    -> "google.com"  
back(100)  -> "leetcode.com"
```

```
actual = leetcode.com  
leetcode.com <-> google.com  
actual = google.com  
... <-> google.com <-> facebook.com  
... <-> facebook.com <-> youtube.com  
actual = youtube.com  
actual retrocede 1 posición  
actual retrocede 1 posición más  
actual avanza 1 posición  
se descarta youtube.com; nuevo nodo tras  
facebook.com  
no hay más "adelante" que linkedin.com  
actual retrocede una posición  
tope en el primer nodo, no vuelve más
```

¿Y la memoria en Python?

Python libera un objeto en cuanto nadie lo referencia (contador de referencias).

```
visit('linkedin.com') tras back(3):  
    facebook.com -> linkedin.com      <- actual  
    youtube.com <-> tiktok.com <-> x.com <- descartadas
```

- Las descartadas se apuntan entre sí: es un **ciclo**. Nunca llega a 0 referencias y espera al recolector de ciclos (gc).
- Es el «memory leak» de Python: no rompe nada, pero la memoria se libera tarde.
- Lo vemos con `NodoDoble.vivos`, el mismo contador de nodos vivos que en C++.

En casa (Ejercicio 4): romper los enlaces → `vaciar()`, `copiar()` y un `visit()` limpio.

Cierre y actividad para casa

Obligatorio en casa (~2 h)

- `reverse()` e `intercalar()` → PruebasReverse, PruebasIntercalar
- `BrowserHistory`: `visit`, `back`, `forward` → `test_historial`
- Ejercicio 4: `vaciar`, `copiar` y `visit` sin ciclos → PruebasMemoria
- `comparativa.py` + `RESPUESTAS.md` (predice antes de medir)
- Juez, 2 problemas: `lista_doble.py` y `envio_historial.py` (lo crea `generar_envio.py`)
- Opcional: `rotar(k)`

Ahora: test individual (20 preguntas, sin internet)

¡Manos al código!

Siguiente práctica: Pilas